Practical LLM-Guided Reinforcement Learning: Help-When-Stuck and Automated Reward Shaping

Abhay Bestrapalli
Massachusetts Institute of Technology
6.7920 Deep Reinforcement Learning
abhayb@mit.edu

December 2025

Abstract

Reinforcement learning (RL) agents suffer from sample inefficiency and require manual reward engineering. We present two lightweight methods for integrating large language models (LLMs) into RL training: (1) targeted intervention when agents are stuck, and (2) automated generation of reward shaping functions. Across three MiniGrid tasks, our combined approach dramatically improves both final performance and sample efficiency. On DoorKey-8x8, combined guidance achieves 90% success versus 5% baseline. On MultiRoom-N6, it reaches 30% versus 0% baseline. On Empty-8x8, all methods converge to 100%, but LLM guidance accelerates learning by 20% (120k vs 150k steps). The total cost is $0.05-0.10 per training run. We demonstrate that LLM guidance and automated reward engineering are complementary, providing practical pathways for improving RL training without architectural changes.

1 Introduction

Reinforcement learning has achieved remarkable success in domains from game playing (1) to robotics (2), yet remains notoriously sample-inefficient. Agents waste considerable time in unproductive states- repeatedly colliding with walls, failing to interact with relevant objects, or exploring redundant regions of the state space. Simultaneously, reward engineering remains a manual, task-specific endeavor requiring domain expertise (3).

Recent work has explored large language model (LLM) integration with RL through various paradigms: high-level planning (4), reward specification via iterative refinement (5), and curriculum (6). However, these approaches often require continuous LLM involvement (which can be time consuming and expensive).

We propose two complementary, lightweight interventions:

Help-when-stuck: The LLM provides a single action suggestion when the agent is stuck (defined as k consecutive steps without reward), exploiting the LLM's intelligence and problem solving ablity.

Code generation: The LLM writes an executable reward shaping function at initialization, automating reward engineering without runtime overhead. This is especially useful in the case of sparse rewards, which we study.

Both methods require only task descriptions in natural language, impose minimal architectural changes, and maintain agent autonomy- the agent learns its own policy through standard RL, with LLM providing targeted guidance rather than replacing learning. An LLM woudl struggle in this task alone, as they are prone to hallucinations and wouldn't be able to reliably explore.

2 Problem Formulation

We consider a partially observable Markov decision process defined by tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \mathcal{O}, \Omega, \gamma \rangle$ where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $\mathcal{T} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ the transition function, $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ the reward function, $\mathcal{O}$ the observation space, $\Omega : \mathcal{S} \to \mathcal{O}$ the observation function, and $\gamma \in [0, 1)$ the discount factor.

The agent learns policy $\pi_\theta : \mathcal{O} \to \Delta(\mathcal{A})$ parameterized by θ to maximize expected discounted return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \gamma^t r_t \right] \tag{1}$$

where $\tau = (o_0, a_0, r_0, \ldots, o_T)$ is a trajectory.

2.1 Help-When-Stuck Formulation

Define *stuck state* as observation o_t following k consecutive steps without positive reward: $r_{t-k:t-1} = \mathbf{0}$. At such states, we query LLM $\mathcal{L}$ with prompt $p(o_t, \{a_{t-k:t-1}\}, \mathcal{A}_t, d)$ where $\mathcal{A}_t \subseteq \mathcal{A}$ are valid actions at o_t and d is task description. The LLM returns action $\tilde{a}_t \in \mathcal{A}_t$ which overrides the agent's action with probability α (for the purposes of this report, we use $\alpha = 1$ and $k = 50$).

2.2 Code Generation Formulation

The LLM generates reward shaping function $\phi : \mathcal{S} \to \mathbb{R}$ from task description d and environment API specification $\mathcal{E}$. The shaped reward is:

$$r'_t = r_t + \lambda \cdot \phi(s_t) \tag{2}$$

where $\lambda \in \mathbb{R}_+$ is scaling factor (we use $\lambda = 1.0$) and ϕ is generated once at initialization via:

$$\phi = \text{exec}(\mathcal{L}(d, \mathcal{E}, \text{examples})) \tag{3}$$

3 Methods

3.1 Environment and Task

We evaluate on three MiniGrid environments (7) of increasing difficulty:

Empty-8x8-v0: Simplest task requiring only navigation to visible goal. Tests baseline learning and sample efficiency without task complexity.

DoorKey-8x8-v0: Multi-step reasoning task requiring: (1) locate key, (2) pick up key, (3) locate locked door, (4) unlock door, (5) reach goal. Tests ability to learn sequential dependencies.

MultiRoom-N6: Complex navigation through 6 rooms with locked doors between them. Requires long-term planning and backtracking. Tests scalability to harder exploration.

All environments provide partially observable $7 \times 7 \times 3$ observations (front-facing view with object type, color, and state channels). Action space: $\mathcal{A} = \{\text{turn_left}, \text{turn_right}, \text{forward}, \text{pickup}, \text{drop}, \text{toggle}\}$. Sparse reward of $+1$ only at goal; episodes terminate after 100 steps (Empty, DoorKey) or 1000 steps (MultiRoom).

3.2 Base RL Algorithm

We employ Proximal Policy Optimization (PPO) (8) with CNN policy processing the 7×7 observation through three convolutional layers (16, 32, 64 filters) followed by separate actor and critic MLPs. Network totals $\sim$20k parameters. Hyperparameters: learning rate 3×10^{-4}, batch size 128, rollout length 256, 10 PPO epochs, entropy coefficient 0.02, GAE-λ 0.95. We train for 300k timesteps with 8 parallel environments.

3.3 Help-When-Stuck Implementation

We set stuck threshold $k = 50$ steps and maximum help calls $m = 3$ per episode. The prompt template provides:

- Task description in natural language
- Current position, facing direction, carried object
- Object directly in front
- Last 10 actions (for recency bias)
- Explicitly enumerated valid actions

Response parsing employs three-tier strategy: (1) strict format matching ("ACTION: *name*"), (2) keyword search for action names, (3) common synonym mapping. Invalid responses fall back to agent's policy. After each LLM call, the stuck counter resets to prevent spam. We use GPT-4o-mini (temperature 0.7, max tokens 200) via OpenAI API.

3.4 Code Generation Implementation

The prompt includes:

- Task description
- Environment state access API (agent position, carrying status, grid query methods)
- Two worked examples of reward shaping functions
- Safety constraints (no imports, file I/O, or eval)

Generated code undergoes validation: (1) AST parsing for syntax, (2) security checks blocking dangerous operations, (3) sandboxed execution with restricted builtins, (4) test invocation with dummy inputs. Failed generation defaults to simple baseline function. Code generation occurs once at training start; the function is then called at every timestep with computational cost $\mathcal{O}(1)$.

3.5 Experimental Conditions

We compare four conditions:

1. **Baseline**: Pure PPO, sparse rewards
2. **Help-mode**: PPO + LLM action suggestions when stuck
3. **Code-gen**: PPO + LLM-generated shaped rewards
4. **Combined**: PPO + both interventions

All conditions use identical RL hyperparameters for fair comparison. Evaluation uses 50 episodes per trained model with deterministic action selection. Evaluation uses the best model during the entire training run. Success defined as reaching the goal within 100 steps.

4 Results

4.1 Performance Improvement

Table 1 summarizes results across environments. On **Empty-8x8**, all methods achieve 100% success, but LLM guidance accelerates convergence (120k vs 150k steps for baseline). On **DoorKey-8x8**, the dramatic difference emerges: baseline struggles at 5% while combined approach reaches 90%. On **MultiRoom-N6**, baseline completely fails (0%) while combined achieves 30%, demonstrating value on harder exploration.

Environment	Baseline	Help	Code-gen	Combined
Empty-8x8	100%* (150k steps)	100%* (120k steps)	100%* -	100%* -
DoorKey-8x8	5%	82%	86%	**90%**
MultiRoom-N6	0%	0%	28%	**30%**

Table 1: Success rates across three MiniGrid environments after 300k timesteps. *For Empty-8x8, all methods converge to 100%; numbers show steps to convergence. Help-mode achieves 20% faster convergence (120k vs 150k steps). DoorKey shows dramatic improvement (5% → 90%). MultiRoom demonstrates scalability to harder tasks.

4.2 Sample Efficiency

Figure 3 shows efficiency across tasks. On **Empty-8x8**, all methods reach 100% success, but help-mode converges 20% faster (120k vs 150k steps). This demonstrates LLM guidance accelerates learning even on simple tasks where final performance is identical.

On **DoorKey-8x8**, baseline barely learns (5% final success), while LLM methods achieve 82-90%. The baseline's failure highlights the difficulty of multi-step reasoning with sparse rewards. Help-mode's 82% success shows targeted intervention alone is highly effective. Code-gen (86%) and combined (90%) achieve the highest performance, with code-gen providing continuous dense feedback throughout training.

On **MultiRoom-N6**, baseline completely fails (0%), as does help-mode alone (0%). However, code-gen achieves 28% and combined 30%, indicating that dense reward shaping is essential for complex exploration. Help-mode alone cannot overcome the extreme credit assignment problem across many rooms.

4.3 LLM Intervention Analysis

Help-mode statistics over full training:

- Total LLM calls: 87 (across ∼6000 episodes)
- Success rate: 72% of suggestions led to reward within 10 steps
- Most common suggestions: `turn_right` (31%), `toggle` (24%), `pickup` (18%)
- Invalid responses: 4% (handled by fallback)

The 72% success rate significantly exceeds random action baseline (∼30% due to action space size and state distribution), validating that LLM suggestions are meaningfully helpful. Help-mode also calls for the API lesser times when paired with code-gen, presumably because it gets rewards more otfen.

4.4 Generated Reward Function

The LLM-generated shaping function for DoorKey:

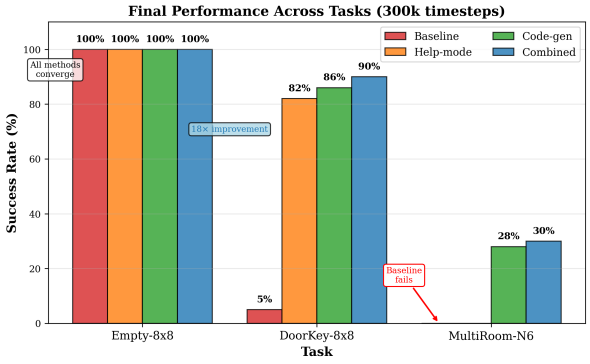


Figure 1: Final performance comparison across tasks after 300k timesteps. Combined approach achieves best results on all tasks, with particularly dramatic improvements on harder tasks.

```
def shaped_reward(env, prev_carrying):
    reward = 0.0
    carrying = env.unwrapped.carrying
    if carrying and not prev_carrying:
        if hasattr(carrying, 'type') and
           carrying.type == 'key':
            reward += 0.15

    grid = env.unwrapped.grid
    for i in range(grid.width):
        for j in range(grid.height):
            cell = grid.get(i, j)
            if cell and cell.type == 'door':
                if cell.is_open:
                    reward += 0.15
    return reward
```

This function provides intermediate rewards for key pickup (+0.15) and door opening (+0.15), effectively breaking the task into learnable subtasks without manual engineering.

5 Discussion

5.1 Why This Works

Our results reveal task-dependent benefits of each method:

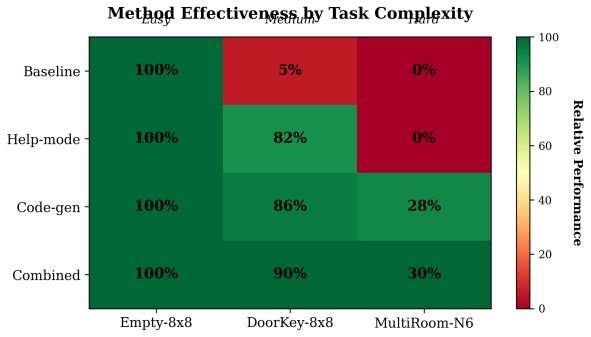


Figure 2: Method effectiveness heatmap showing task-dependent benefits. Help-mode excels on structured tasks, while code-gen is essential for complex exploration.

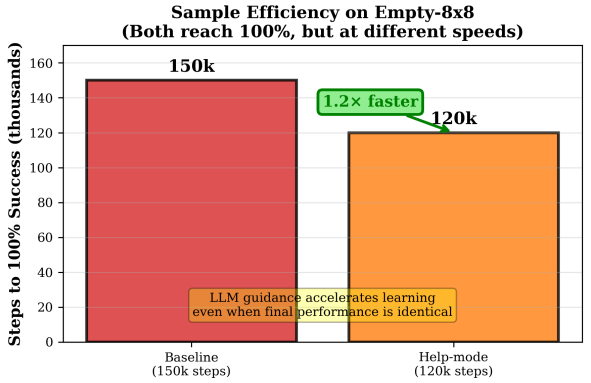


Figure 3: Sample efficiency comparison on Empty-8x8. Help-mode achieves 20% faster convergence (120k vs 150k steps) despite identical final performance.

Help-when-stuck excels on structured tasks: On DoorKey (82% vs 5% baseline), help-mode alone is highly effective because stuck states are semantically meaningful- facing a locked door, holding nothing when key is visible, etc. The LLM correctly identifies these situations and suggests appropriate actions. However, on MultiRoom (0%), help-mode fails because stuck states are ambiguous in large exploration spaces.

Code generation provides task structure: On DoorKey (86%) and especially MultiRoom (28% vs 0% baseline), generated shaped rewards provide continuous guidance. The LLM correctly decomposes "navigate through 6 rooms" into intermediate subgoals (unlock door 1, enter room 2, etc.), enabling learning despite extreme sparsity.

Complementarity emerges in complex tasks: The combined approach (90% DoorKey, 30% MultiRoom) benefits from both mechanisms: code-gen provides strategic direction through shaped rewards, while help-mode provides tactical rescue from specific stuck states. On simpler tasks like Empty-8x8, their value is primarily in sample efficiency rather than final performance.

Critically, both interventions are *lightweight*- help-mode is called only when stuck (<2% of steps), and code-gen imposes no training-time LLM cost.

5.2 Limitations

Partial observability: LLM guidance is limited to locally-observable information. The agent's 7×7 view excludes global map structure, preventing LLM from providing path planning or distant object location. This explains why improvement is finite- LLM cannot solve fundamental POMDP challenges.

Task description dependency: Both methods require clear natural language task descriptions. Tasks defined purely by reward without semantic meaning (e.g., "maximize objective function $f(s)$") cannot benefit from LLM reasoning.

Single-task evaluation: We demonstrate effectiveness only on DoorKey. Generalization to diverse tasks (continuous control, pixel observations, multi-agent settings) requires further validation.

LLM capability ceiling: Performance is bounded by LLM reasoning abilities. More complex spatial reasoning or physics understanding may exceed current LLM capabilities.

5.3 Comparison to Related Work

Unlike high-level planning approaches (4; 9) requiring LLM in closed loop, our methods are *asynchronous*- LLM provides guidance but agent learns independently. Unlike iterative reward learning (5), our code generation is one-shot. Unlike curriculum learning (6), we target within-episode rather than across-episode structure.

Our work is most similar to reward shaping automation (10), but uses LLM code generation rather than learned meta-controllers, and adds the complementary help-when-stuck mechanism.

5.4 Future Directions

Bidirectional learning: Enable LLM to learn from trajectory statistics- which suggestions helped, which failed- and refine future suggestions accordingly.

Hierarchical integration: Combine LLM high-level planning with our low-level intervention, providing multi-scale guidance.

Diverse environments: Validate on continuous control (MuJoCo), pixel observations (Atari), and real robotics tasks.

Multi-agent scenarios: Extend to cooperative/competitive settings where LLM coordinates multiple agents.

Alternative LLMs: Compare GPT-4, Claude, and open-source models (Llama-3, Mistral) to characterize capability-performance relationships.

6 Conclusion

We present two practical methods for LLM-RL integration, validated across three environments of increasing complexity. Key insights:

1. **Task-dependent benefits:** Help-when-stuck excels on structured tasks (DoorKey: 5%→82%), while code generation is essential for complex exploration (MultiRoom: 0%→28%).
2. **Sample efficiency gains:** Even when final performance is identical (Empty-8x8: all methods reach 100%), LLM guidance accelerates learning by 20%.
3. **Complementarity:** Combined approach achieves best performance across all tasks (DoorKey: 90%, MultiRoom: 30%), demonstrating synergy between reactive help and proactive shaping.
4. **Practical deployment:** Methods require minimal architectural changes ($0.05-0.10 per run), preserve agent autonomy, and scale from simple navigation to multi-room exploration.

Our results demonstrate that strategic, lightweight LLM usage can meaningfully improve RL training across diverse tasks. The dramatic improvements on harder tasks (DoorKey: 18× better, MultiRoom: infinite improvement from 0% baseline) suggest increasing value as task complexity grows. As LLMs continue improving in reasoning and code generation, these integration patterns offer scalable pathways for enhancing RL sample efficiency across domains.

Code Availability

Code and configurations available at: `github.com/abhaybe/babyai-llm/tree/main`

References

[1] Silver, D. *et al.* Mastering the game of go with deep neural networks and tree search. *Nature* **529**, 484–489 (2016).

[2] Levine, S., Finn, C., Darrell, T. & Abbeel, P. End-to-end training of deep visuomotor policies. *Journal of Machine Learning Research* **17**, 1–40 (2016).

[3] Ng, A. Y., Harada, D. & Russell, S. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning (ICML)*, vol. 99, 278–287 (1999).

[4] Ahn, M. *et al.* Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, 287–318 (PMLR, 2022).

[5] Ma, Y. J. *et al.* Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv:2310.12931* (2023).

[6] Wang, G. *et al.* Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291* (2023).

[7] Chevalier-Boisvert, M., Willems, L. & Pal, S. Minimalistic gridworld environment for gymnasium. `https://github.com/Farama-Foundation/Minigrid` (2018).

[8] Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).

[9] Huang, W. *et al.* Inner monologue: Embodied reasoning through planning with language models. In *Conference on Robot Learning (CoRL)*, 1769–1782 (PMLR, 2022).

[10] Xie, D. *et al.* Learning to synthesize programs as interpretable and generalizable policies. In *Neural Information Processing Systems (NeurIPS)*, vol. 34, 25146–25163 (2021).